

Tomboul-Rosette Joshua
El-Chawa Lara
Craig Cassar

4A AE FISA



Programmation Orientée Objet

BE: AudioMixer

Lien du projet GitHub:

https://github.com/lara-elchawa/BE_POO_Audio_Mixer.git

Sommaire:

Contains:	2
1. AudioMixer	3
1.1. Définition	3
1.2. Composants	3
1.3. Diagramme de classe de notre projet global	3
2. Réalisation du projet Mixeur Audio	4
2.1. Transmission de l'état du volume (branch PC-ESP):	4
2.2. Affichage de l'application (branch LCD):	4
2.3. Modification du niveau de son (branch Encoder):	5
2.4. LEDS RGB	5
2.5. Communication ordinateur-AudioMixer	5

1. AudioMixer

1.1. Définition

L'AudioMixer est une interface physique permettant de modifier le volume des pistes audio de n'importe quel ordinateur windows 11 via une liaison usb classique. L'AudioMixer permet également de manipuler plus précisément les pistes audio de l'ordinateur :

- Mise en pause de la piste audio
- Coupure du son de la piste audio
- Prochaine/précédente musique/vidéo

1.2. Composants

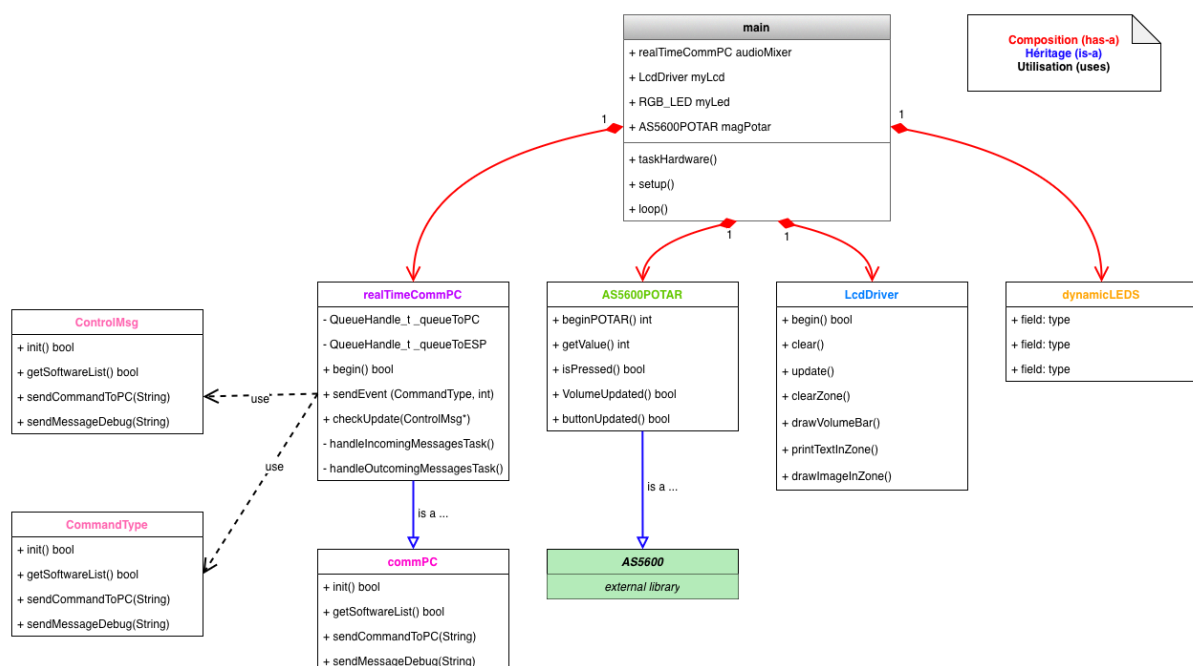
LCD screen	0.96 pouces oled I2C
Encodeur	AS5600
Boutons poussoirs	Au choix
LED RGB	WS2811B
USBC	4 pins USB 3,1 Type C
Multiplexeur I2C	TCA9548A

1.3. Diagramme de classe de notre projet global

Composition (Structure) : La classe **main** est le cœur du projet. Elle possède une relation de composition avec les composants (**Écran**, **Encodeur**, **Leds**, **CommPC**) car leurs instances sont créées et gérées par le main. Toute l'information transite par cette structure centrale.

Les attributs sont **private** ou **protected**. Cela garantit que le matériel est piloté de manière sécurisée via les méthodes publiques.

Communication : La classe **realTimeCommPC** gère le dialogue bidirectionnel avec le script Python sur le PC via le port Série.



2. Réalisation du projet Mixeur Audio

2.1. Transmission de l'état du volume (branch PC-ESP):

L'utilisateur interagit avec un encodeur rotatif. L'ESP32 traite ce mouvement et envoie instantanément un ordre numérique au PC. Un script Python sur le PC reçoit cet ordre et modifie le volume du système ou de l'application active (Spotify, YouTube, etc.).

```
Réponse 'OK' envoyée à l'ESP
L'ESP dit : CHECK_STATUS
Réponse 'OK' envoyée à l'ESP
L'ESP dit : CHECK_STATUS
Réponse 'OK' envoyée à l'ESP
L'ESP dit : CHECK_STATUS
Réponse 'OK' envoyée à l'ESP
```

2.2. Affichage de l'application (branch LCD):

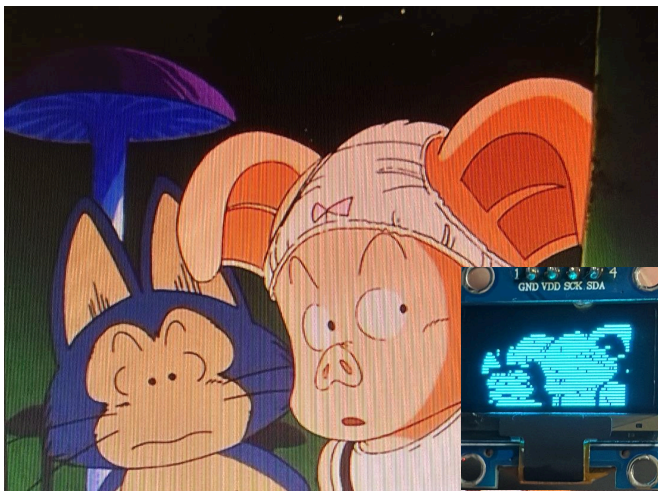
Notre objectif pour le LCD est de pouvoir afficher les différents noms, volume actuel, un logo et la couleur des applications que l'on utilise pour l'audio.

Pour cela, le PC envoie des informations à l'ESP32 pour mettre à jour l'affichage LCD et les LEDs:

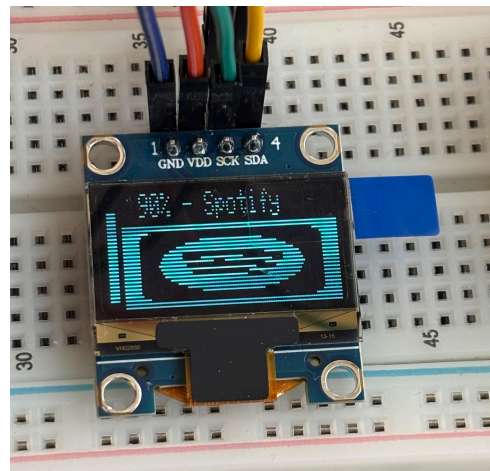
- Tant que le script (python) PC n'est pas connecté à l'ESP32, l'écran affiche "**Disconnected**". Dès que la liaison est établie, il affiche "**Connected**".
- Le PC transmet alors le nom de l'application qui diffuse du son (ex: "*Spotify*" ou "*YouTube*").
- L'ESP32 affiche le nom de l'application sur le LCD et la valeur du volume actif.
- En fonction de l'application détectée, l'ESP32 change la couleur des **LEDs WS2812B** pour correspondre à l'identité visuelle de l'application (ex: **Vert** pour *Spotify*, **Rouge** pour *YouTube*).

→ Actuellement, les leds, le nom et le volume s'affichent correctement, seulement le logo n'est pas encore implémenté.

→ Avant d'intégrer le LCD avec le reste du projet, nous l'avons testé unitairement en deux étapes :



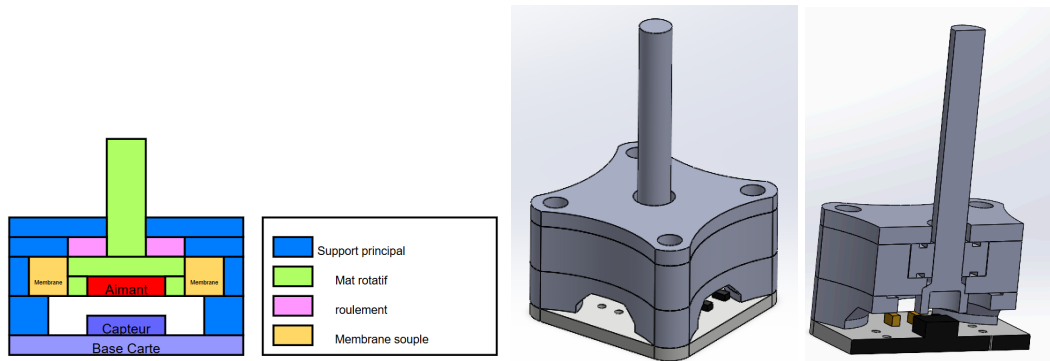
Test n°1: est-il possible d'afficher une image ? Oui !



Test n°2: est-il possible de scinder l'écran en 3 parties? Oui !

2.3. Modification du niveau de son (branch Encoder):

L'encodeur magnétique permet de contrôler précisément le volume du son de l'AudioMixer en communiquant en I2C avec l'ESP32. Il est ainsi composé :



Une puce AS5600 détecte la rotation du champ magnétique d'un aimant à magnétisation diamétrale. L'alignement entre l'aimant et le capteur AS5600 doit être précis, c'est pourquoi nous avons modélisé l'encodeur magnétique sur SOLIDWORKS en prenant soin d'aligner précisément le mât rotatif et le capteur. La puce AS5600 intègre un capteur de force de champ magnétique, nous en avons profité pour ajouter une fonction "bouton poussoir", en gérant soigneusement les tolérances diamétrales entre le mât rotatif et le roulement à bille, le mât rotatif peut bouger transversalement en gardant un bon maintien à la structure globale de l'encodeur et des frictions réduites :



Ce montage nous permet de contrôler avec précision le volume des pistes de l'ordinateur. Toutes les caractéristiques de sensibilité peuvent être configurées, l'encodeur magnétique peut donc être configuré de beaucoup de manières différentes en fonction des préférences de l'utilisateur.

2.4. LEDS RGB

L'AudioMixer intègre des LEDs RGB adressables pouvant changer de couleur en fonction de la couleur du logiciel actuellement utilisé. Le code couleur du logiciel est transmis par l'ordinateur, l'AudioMixer n'associe pas des codes couleurs en mémoire à un nom de logiciel. Les LEDs sont adressables donc il est possible d'en mettre énormément. Pour des raisons de limitations d'alimentation sur le port USB standard d'un ordinateur, nous avons installé 3 LEDs.

2.5. Communication ordinateur-AudioMixer (branch PC-ESP)

Un script Python crée une IHM pour visualiser l'état de la connexion et lister les applications audio détectées. Il se synchronise avec l'ESP32 via un handshake en liaison série qu'il initie. Après avoir initié le handshake, le script peut recevoir des messages de commandes de l'ESP, il enverra à chaque fois un message d'acknowledge pour confirmer à l'ESP32 que les commandes ont bien été appliquées. Si l'ESP32 ne reçoit pas d'"ACK", il tente l'opération 3 fois, au bout de 3 essais, la commande est abandonnée et on considère un "timeout".

La liaison avec le PC est possible via les fonctions de la bibliothèque de base "commPC", mais nous avons également mis au point une version "realTimeCommPC", qui passe la communication avec le PC en temps réel via FRTOS, en dédiant un coeur de l'ESP32 au polling et à la gestion des capteurs, et un coeur à la communication avec le PC via deux tâches. Ces tâches communiquent via des files d'attente :

- Une file d'attente pour envoyer des messages au PC
- Une file d'attente pour gérer les messages reçus par le PC (pas utilisé dans notre cas mais par nous l'avons intégré au cas où)

La séparation des deux coeurs nous a permis de rendre le programme globalement non bloquant, car la communication avec le PC est bloquante (gestion des ACK/timeout etc...). Même le coeur 1 attend une réponse du PC, le coeur 0 envoie quand même des messages sur la file d'attente du coeur 1, qui les enverra au PC dès que les messages bloquants seront traités. On ne risque pas de saturer la file d'attente car la liaison série avec l'ordinateur est cadencée à un baud rate de 921600, et que nous avons alloué assez d'espace mémoire pour ne pas saturer l'ESP32.

Nous aurions pu mieux gérer cette communication en temps réel mais nous n'avons pas eu le temps.

2.6. Conclusion

Le projet est fonctionnel, mais nous n'avons pas eu le temps d'imprimer le boîtier d'intégration pour le mixeur audio et de graver la carte électronique :

